LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Sartika Wahidiyati 2609106058

Kelas B2 ‘26

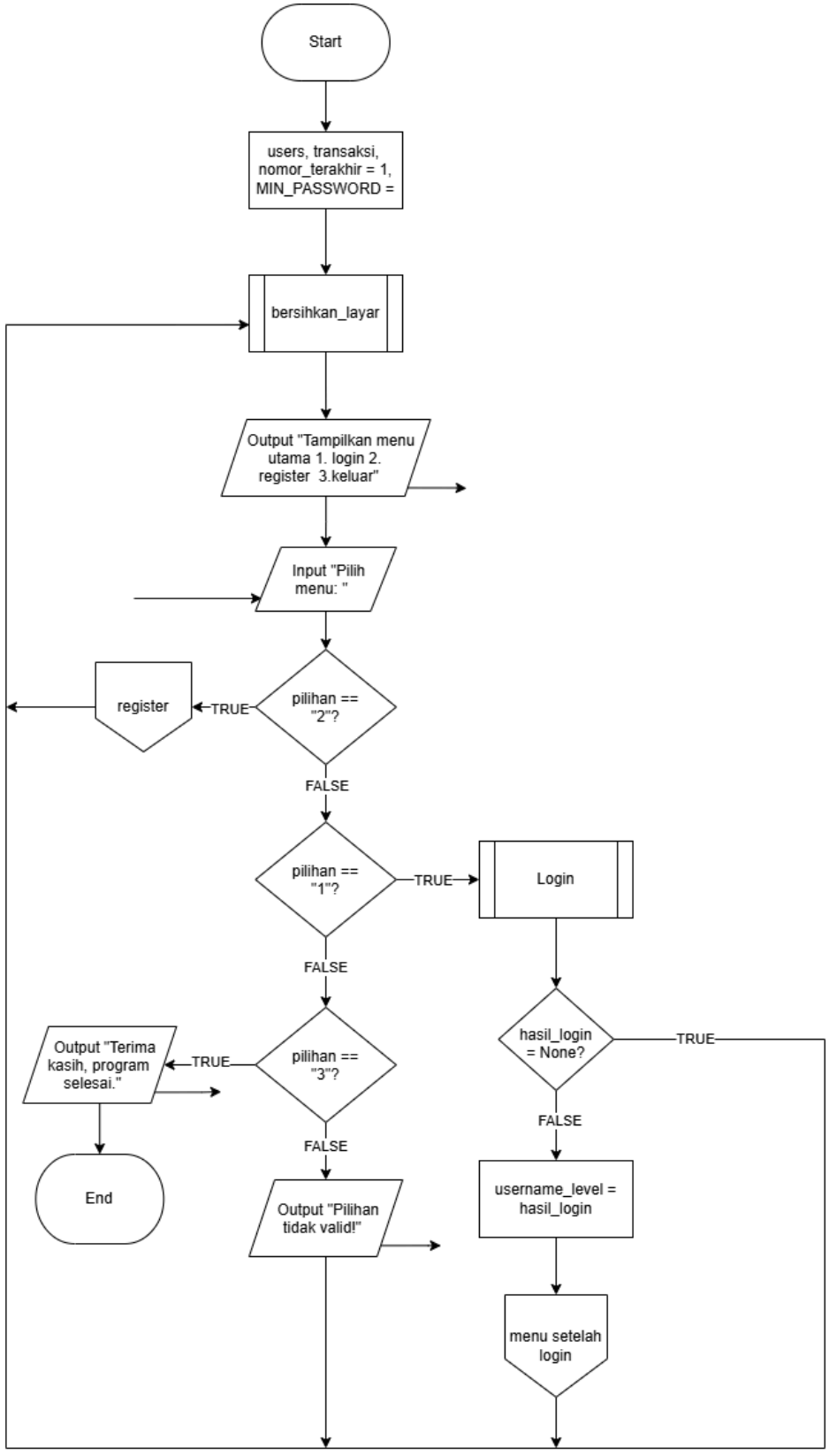
PROGRAM STUDI INFORMATIKA

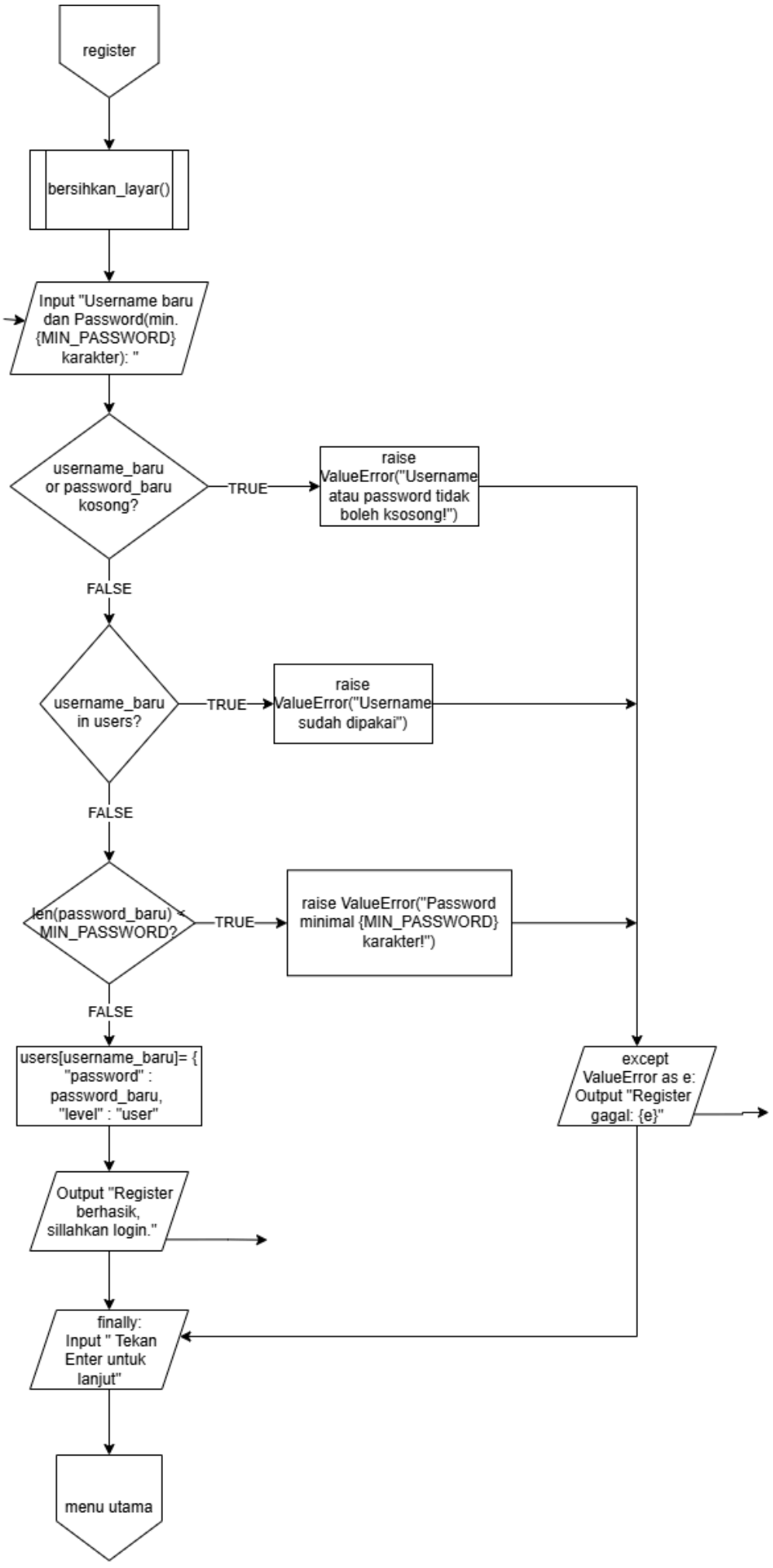
UNIVERSITAS MULAWARMAN

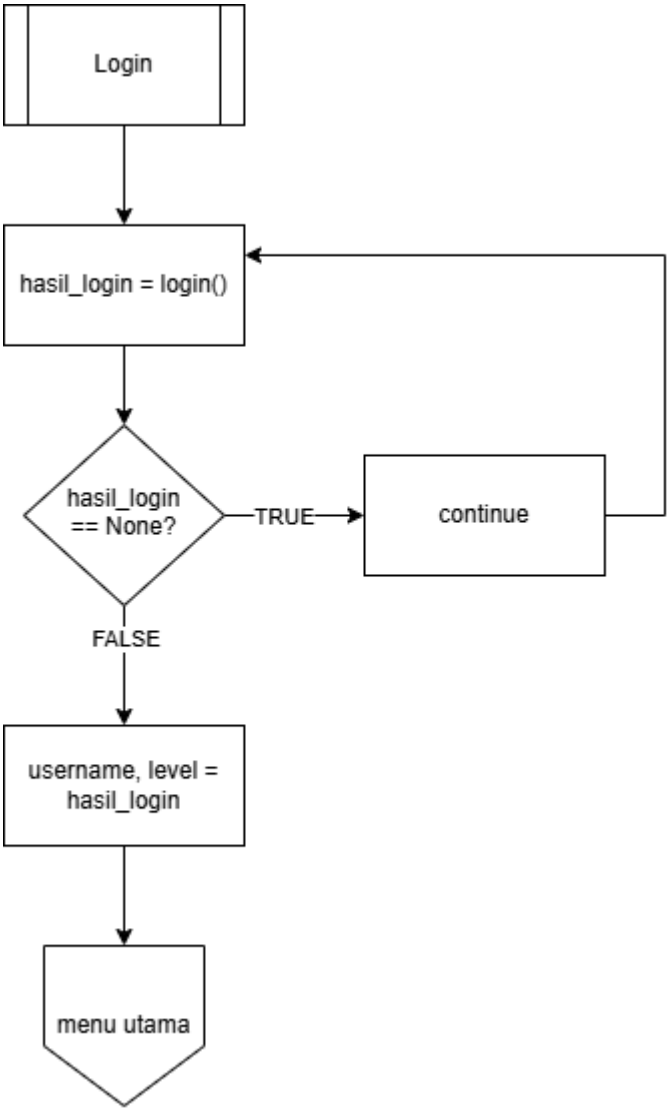
SAMARINDA

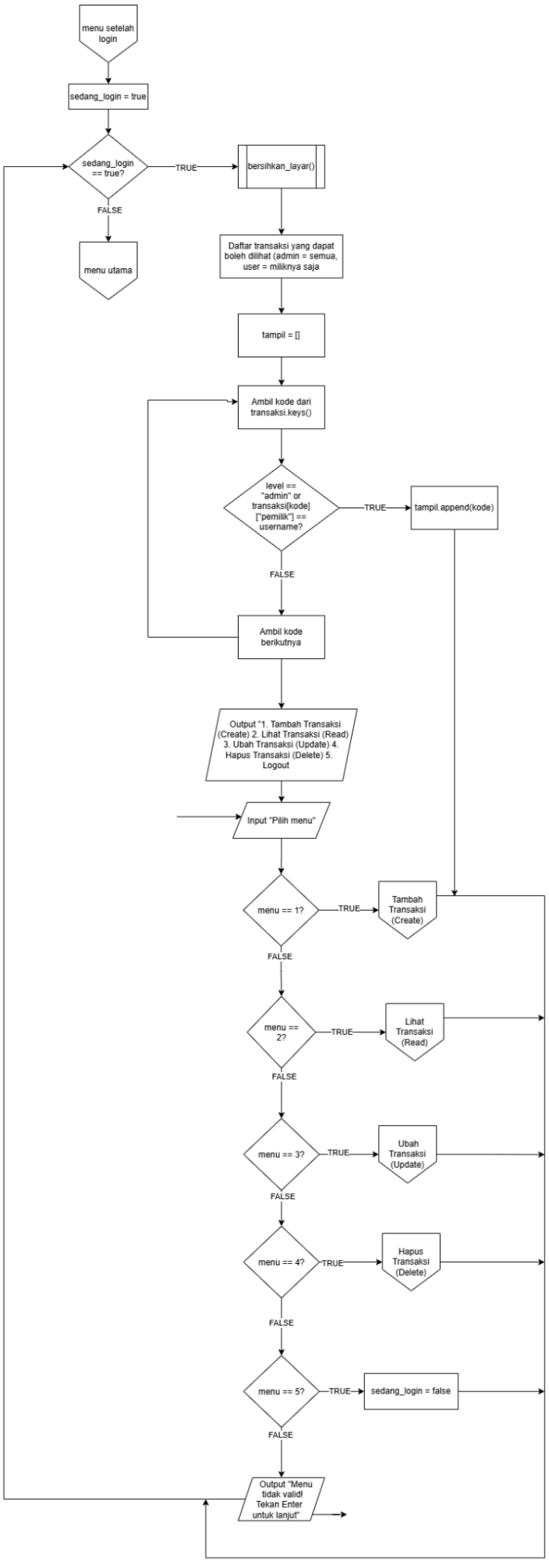
202

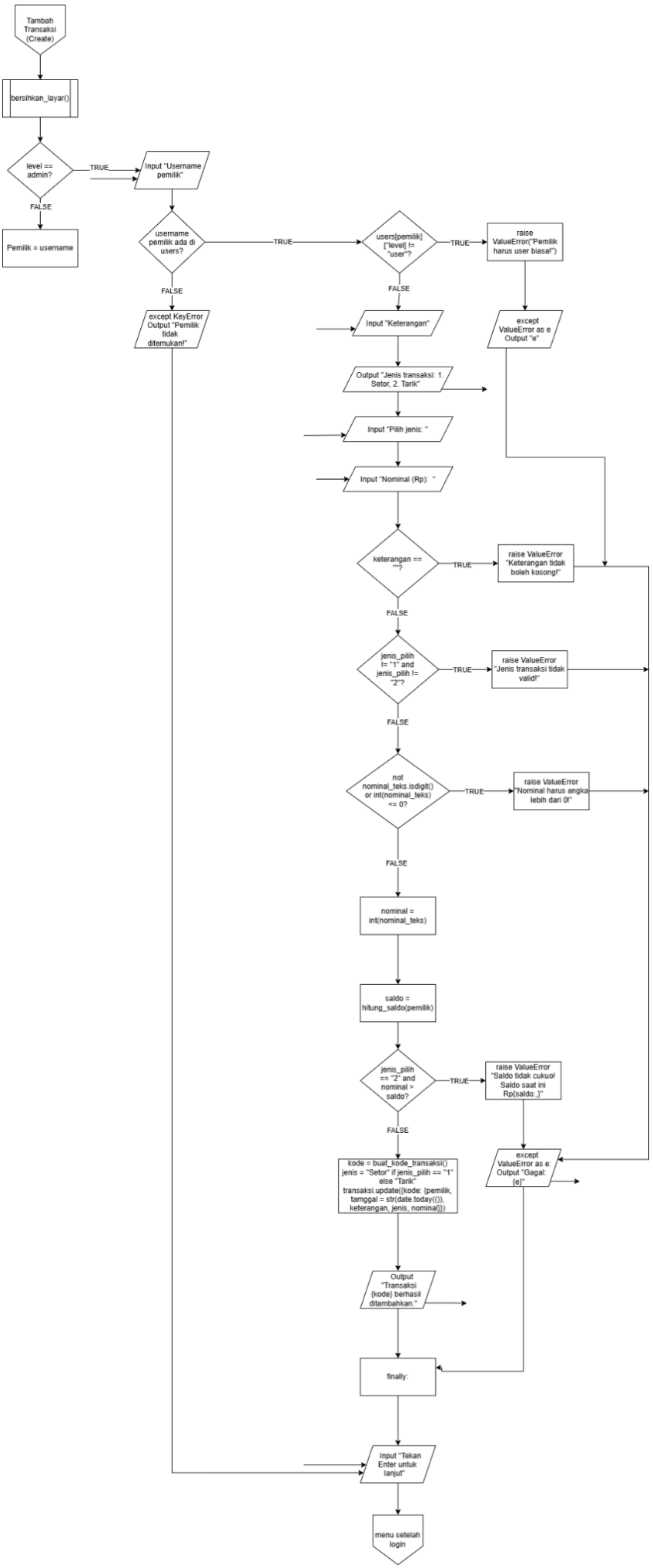
1. Flowchart

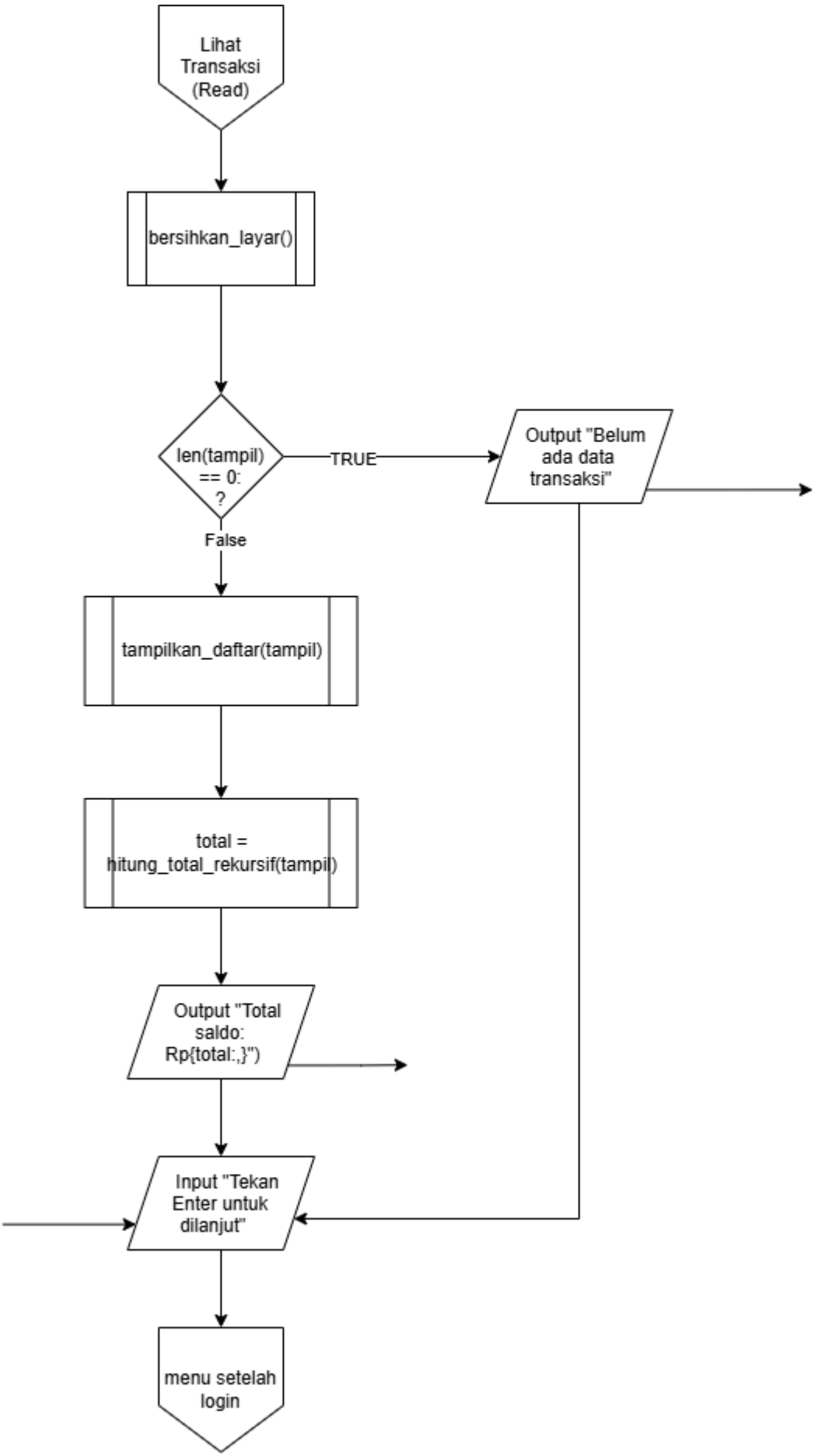

Start
users, transaksi, nomor_terakhir = 1, MIN_PASSWORD =
bersihkan_layar
Output "Tampilkan menu utama 1. login 2. register 3.keluar"
Input "Pilih menu: "
pilihan == "2"?
TRUE
register
FALSE
pilihan == "1"?
TRUE
Login
FALSE
pilihan == "3"?
TRUE
Output "Terima kasih, program selesai."
End
FALSE
Output "Pilihan tidak valid!"
hasil_login = None?
TRUE
FALSE
username_level = hasil_login
menu setelah login

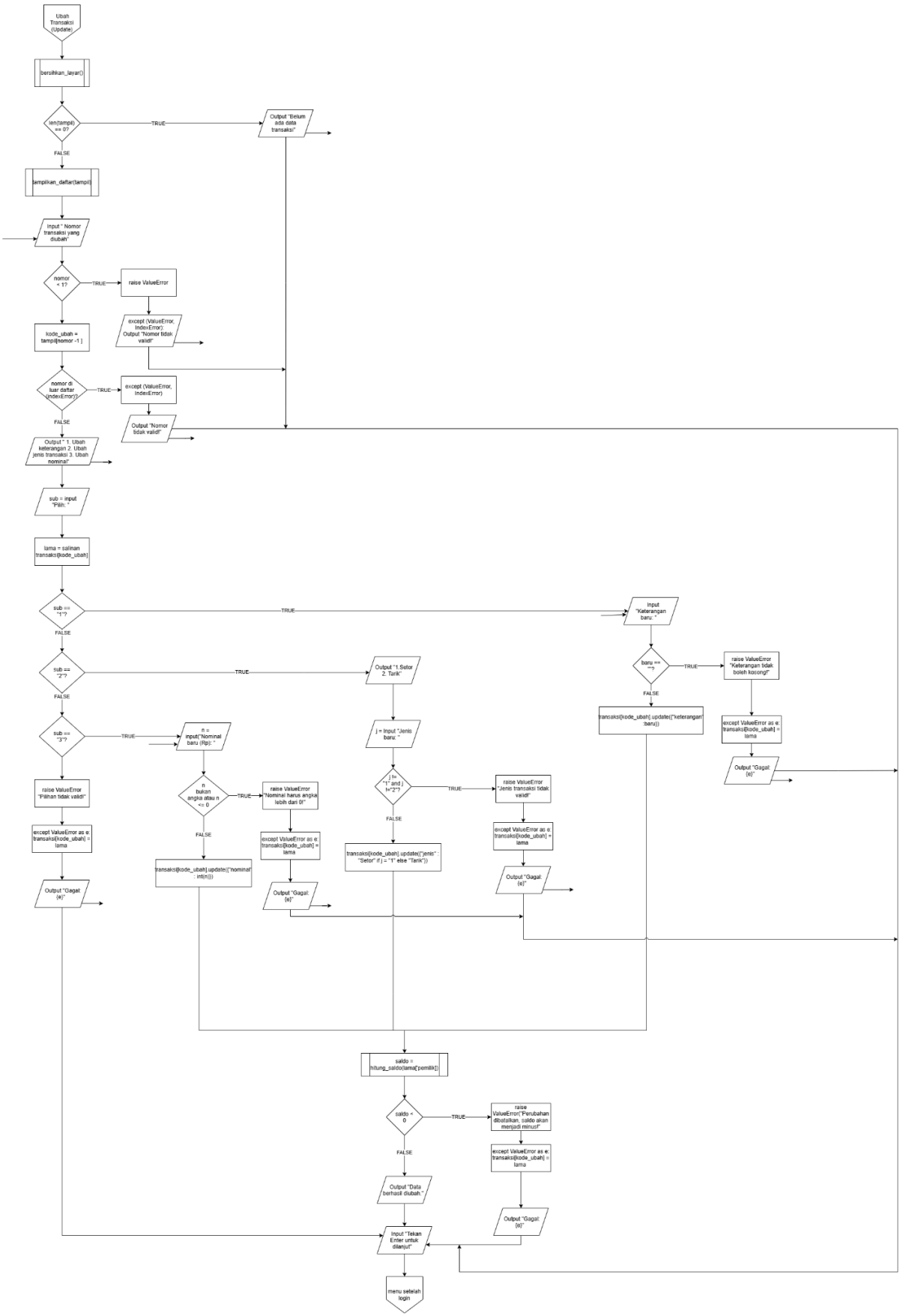

register
bersihkan_layar()
Input "Username baru dan Password(min. {MIN_PASSWORD} karakter): "
username_baru or password_baru kosong?
TRUE
raise ValueError("Username atau password tidak boleh ksosong!")
FALSE
username_baru in users?
TRUE
raise ValueError("Username sudah dipakai")
FALSE
len(password_baru) < MIN_PASSWORD?
TRUE
raise ValueError("Password minimal {MIN_PASSWORD} karakter!")
FALSE
users[username_baru]= { "password" : password_baru, "level" : "user"
except ValueError as e: Output "Register gagal: {e}"
Output "Register berhasik, sillahkan login."
finally: Input " Tekan Enter untuk lanjut"
menu utama

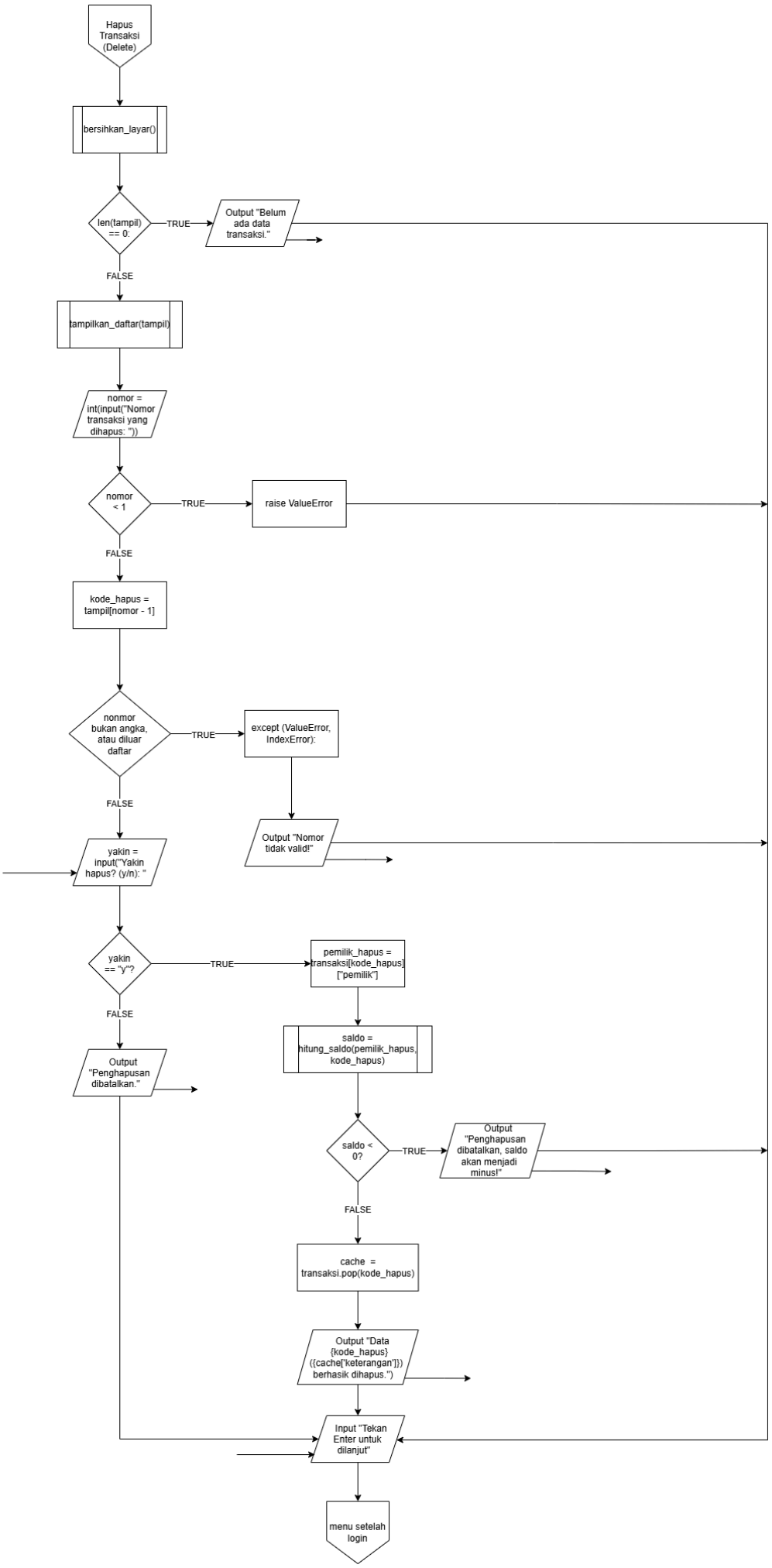

Login
hasil_login = login()
hasil_login == None?
TRUE
continue
FALSE
username, level = hasil_login
menu utama

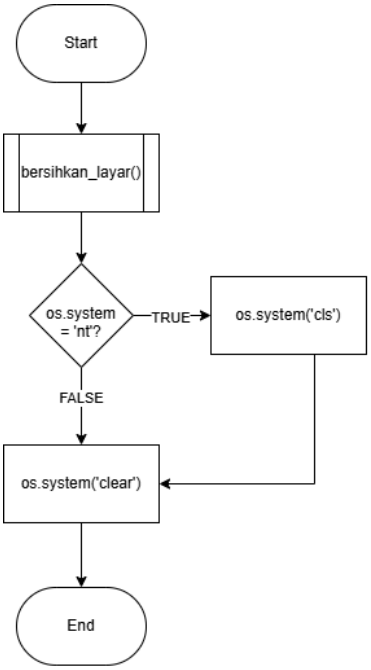

menu setelah login
sedang_login = true
sedang_login == true?
TRUE
FALSE
menu utama
bersihkan_layar()
Daftar transaksi yang dapat boleh dilihat (admin = semua, user = miliknya saja
tampil = []
Ambil kode dari transaksi.keys()
level == "admin" or transaksi[kode] ["pemilik"] == username?
TRUE
tampil.append(kode)
FALSE
Ambil kode berikutnya
Output "1. Tambah Transaksi (Create) 2. Lihat Transaksi (Read) 3. Ubah Transaksi (Update) 4. Hapus Transaksi (Delete) 5. Logout
Input "Pilih menu"
menu == 1?
TRUE
Tambah Transaksi (Create)
FALSE
menu == 2?
TRUE
Lihat Transaksi (Read)
FALSE
menu == 3?
TRUE
Ubah Transaksi (Update)
FALSE
menu == 4?
TRUE
Hapus Transaksi (Delete)
FALSE
menu == 5?
TRUE
sedang_login = false
FALSE
Output "Menu tidak valid! Tekan Enter untuk lanjut"

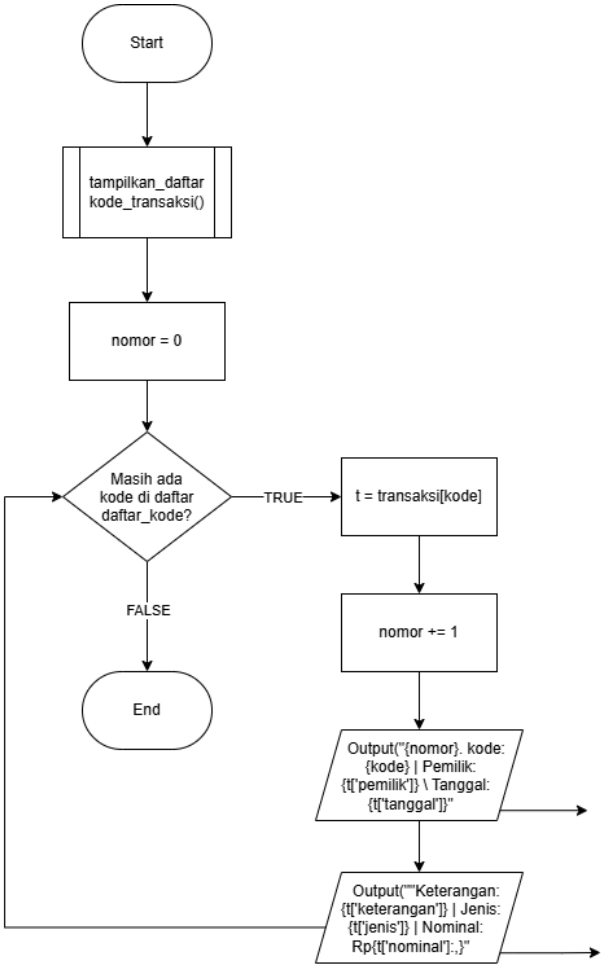

Tambah Transaksi (Create)
bersihkan_layar()
level == admin?
TRUE
Input "Username pemilik"
FALSE
Pemilik = username
username pemilik ada di users?
TRUE
users[pemilik]["level] != "user"?
TRUE
raise ValueError("Pemilik harus user biasa!")
except ValueError as e Output "e"
FALSE
except KeyError Output "Pemilik tidak ditemukan!"
Input "Keterangan"
Output "Jenis transaksi: 1. Setor, 2. Tarik"
Input "Pilih jenis: "
Input "Nominal (Rp): "
keterangan == ""?
TRUE
raise ValueError "Keterangan tidak boleh kosong!"
FALSE
jenis_pilih != "1" and jenis_pilih != "2"?
TRUE
raise ValueError "Jenis transaksi tidak valid!"
FALSE
not nominal_teks.isdigit() or int(nominal_teks) <= 0?
TRUE
raise ValueError "Nominal harus angka lebih dari 0!"
FALSE
nominal = int(nominal_teks)
saldo = hitung_saldo(pemilik)
jenis_pilih == "2" and nominal > saldo?
TRUE
raise ValueError "Saldo tidak cukup! Saldo saat ini Rp{saldo:,}"
FALSE
except ValueError as e: Output "Gagal: {e}"
kode = buat_kode_transaksi() jenis = "Setor" if jenis_pilih == "1" else "Tarik" transaksi.update({kode: {pemilik, tanggal = str(date.today()), keterangan, jenis, nominal}})
Output "Transaksi {kode} berhasil ditambahkan."
finally:
Input "Tekan Enter untuk lanjut"
menu setelah login

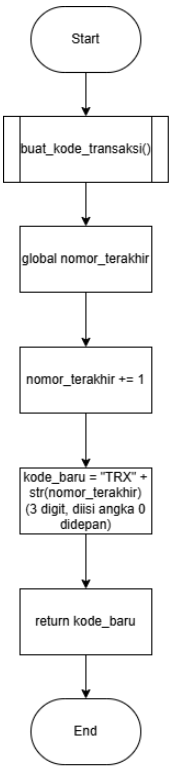

Lihat Transaksi (Read)
bersihkan_layar()
len(tampil) == 0: ?
TRUE
Output "Belum ada data transaksi"
False
tampilkan_daftar(tampil)
total = hitung_total_rekursif(tampil)
Output "Total saldo: Rp{total:,}")
Input "Tekan Enter untuk dilanjut"
menu setelah login

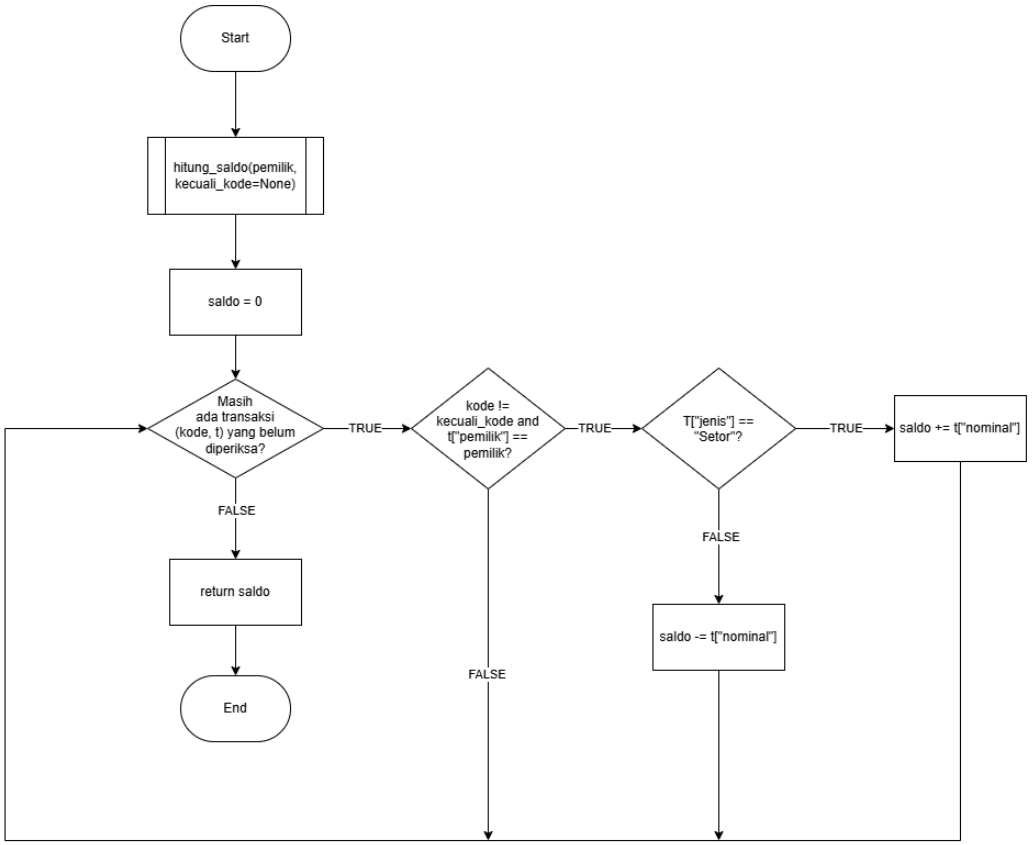

Ubah Transaksi (Update)
bersihkan_layar()
len(tampil) == 0?
TRUE
Output "Belum ada data transaksi"
FALSE
tampilkan_daftar(tampil)
input " Nomor transaksi yang diubah"
nomor < 1?
TRUE
raise ValueError
except (ValueError, IndexError): Output "Nomor tidak valid!"
kode_ubah = tampil[nomor -1]
nomor di luar daftar (IndexError)?
TRUE
except (ValueError, IndexError)
Output "Nomor tidak valid!"
FALSE
Output " 1. Ubah keterangan 2. Ubah jenis transaksi 3. Ubah nominal"
sub = input "Pilih : "
lama = salinan transaksi[kode_ubah]
sub == "1"?
TRUE
input "Keterangan baru: "
baru == ""?
TRUE
raise ValueError "Keterangan tidak boleh kosong!"
except ValueError as e: transaksi[kode_ubah] = lama
Output "Gagal: {e}"
FALSE
transaksi[kode_ubah].update({"keterangan" :baru})
sub == "2"?
TRUE
Output "1.Setor 2. Tarik"
j = input "Jenis baru: "
j != "1" and j != "2"?
TRUE
raise ValueError "Jenis transaksi tidak valid!"
except ValueError as e: transaksi[kode_ubah] = lama
Output "Gagal: {e}"
FALSE
transaksi[kode_ubah].update({"jenis" : "Setor" if j = "1" else "Tarik"})
sub == "3"?
TRUE
n = input("Nominal baru (Rp): "
n bukan angka atau n <= 0
TRUE
raise ValueError "Nominal harus angka lebih dari 0!"
except ValueError as e: transaksi[kode_ubah] = lama
Output "Gagal: {e}"
FALSE
transaksi[kode_ubah].update({"nominal" : int(n)})
raise ValueError "Pilihan tidak valid!"
except ValueError as e: transaksi[kode_ubah] = lama
Output "Gagal: {e}"
saldo = hitung_saldo(lama["pemilik"])
saldo < 0
TRUE
raise ValueError("Perubahan dibatalkan, saldo akan menjadi minus!"
except ValueError as e: transaksi[kode_ubah] = lama
Output "Gagal: {e}"
FALSE
Output "Data berhasil diubah."
input "Tekan Enter untuk dilanjut"
menu setelah login

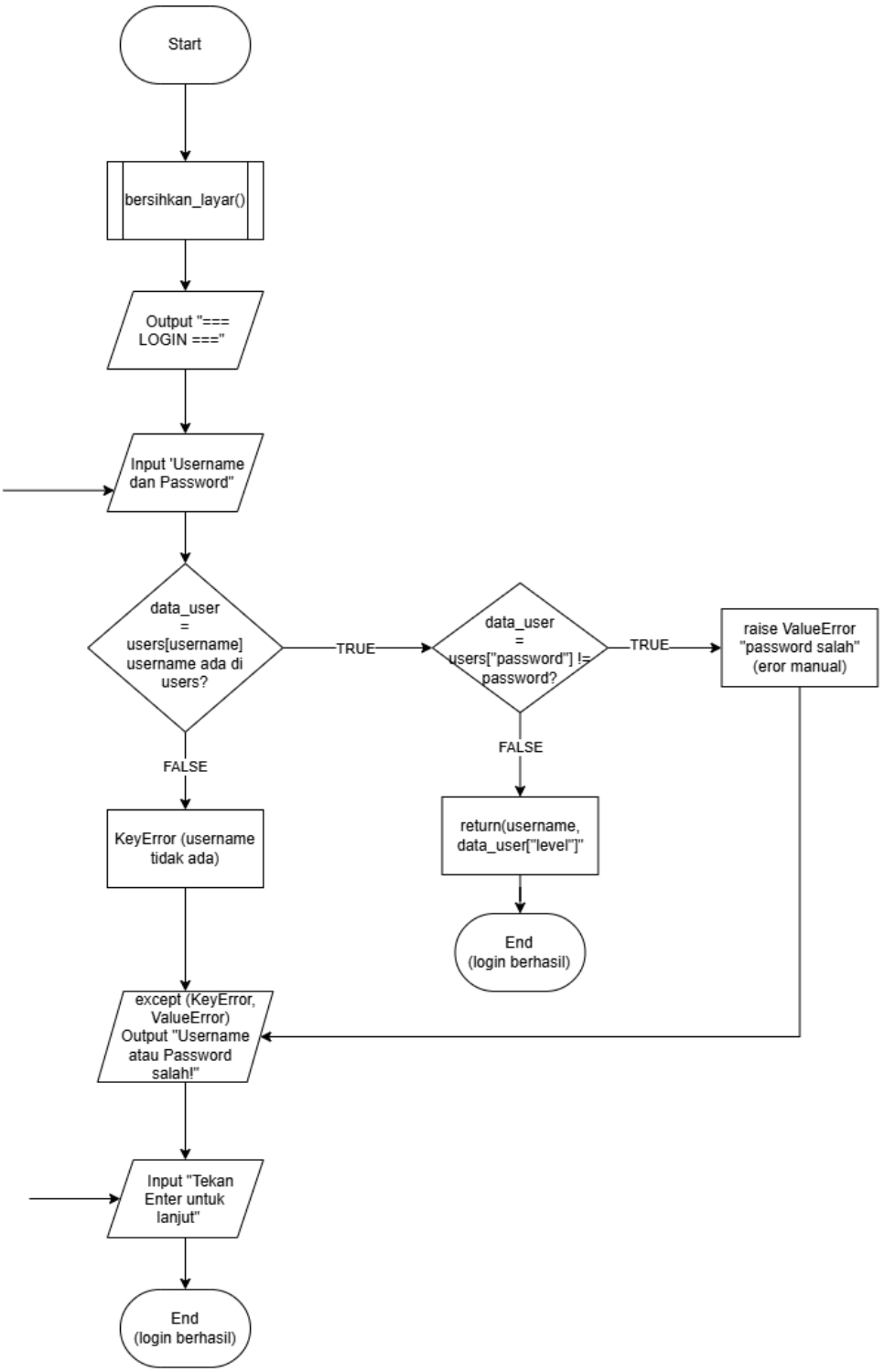

Hapus Transaksi (Delete)
bersihkan_layar()
len(tampil) == 0:
TRUE
Output "Belum ada data transaksi."
FALSE
tampilkan_daftar(tampil)
nomor = int(input("Nomor transaksi yang dihapus: "))
nomor < 1
TRUE
raise ValueError
FALSE
kode_hapus = tampil[nomor - 1]
nonmor bukan angka, atau diluar daftar
TRUE
except (ValueError, IndexError):
Output "Nomor tidak valid!"
FALSE
yakin = input("Yakin hapus? (y/n): "
yakin == "y"?
TRUE
pemilik_hapus = transaksi[kode_hapus]["pemilik"]
FALSE
Output "Penghapusan dibatalkan."
saldo = hitung_saldo(pemilik_hapus, kode_hapus)
saldo < 0?
TRUE
Output "Penghapusan dibatalkan, saldo akan menjadi minus!"
FALSE
cache = transaksi.pop(kode_hapus)
Output "Data {kode_hapus} ({cache['keterangan']}) berhasik dihapus.")
Input "Tekan Enter untuk dilanjut"
menu setelah login

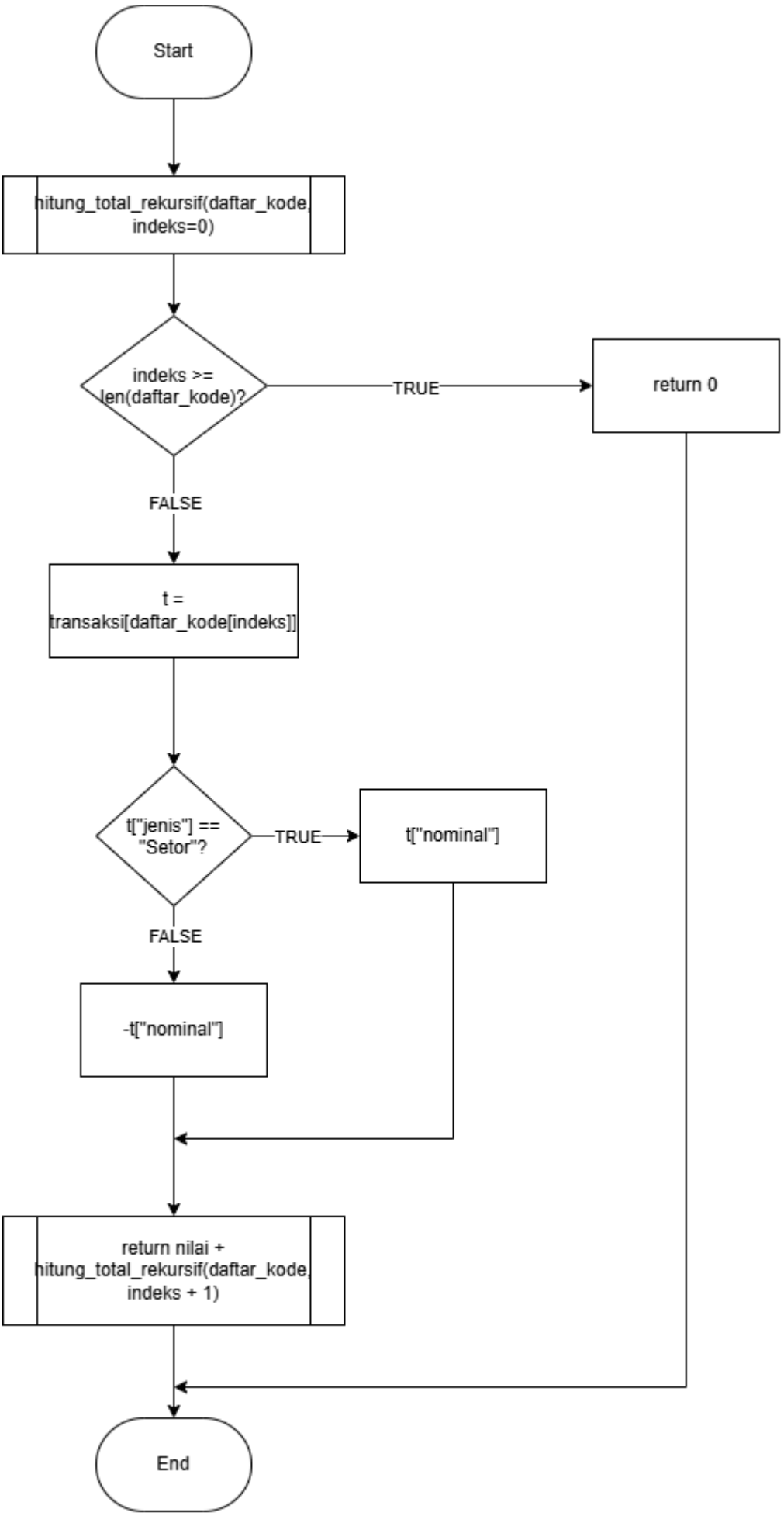

Start
bersihkan_layar()
os.system = 'nt'?
TRUE
os.system('cls')
FALSE
os.system('clear')
End


Start
tampilkan_daftar kode_transaksi()
nomor = 0
Masih ada kode di daftar daftar_kode?
TRUE
t = transaksi[kode]
FALSE
nomor += 1
End
Output("{nomor}. kode: {kode} | Pemilik: {t['pemilik']} \ Tanggal: {t['tanggal']}"
Output(""Keterangan: {t['keterangan']} | Jenis: {t['jenis']} | Nominal: Rp{t['nominal']:,}"


Start
buat_kode_transaksi()
global nomor_terakhir
nomor_terakhir += 1
kode_baru = "TRX" + str(nomor_terakhir) (3 digit, diisi angka 0 didepan)
return kode_baru
End


Start
hitung_saldo(pemilik, kecuali_kode=None)
saldo = 0
Masih ada transaksi (kode, t) yang belum diperiksa?
TRUE
kode != kecuali_kode and t["pemilik"] == pemilik?
TRUE
T["jenis"] == "Setor"?
TRUE
saldo += t["nominal"]
FALSE
return saldo
End
FALSE
saldo -= t["nominal"]
FALSE


Start
bersihkan_layar()
Output "=== LOGIN ==="
Input 'Username dan Password"
data_user = users[username] username ada di users?
TRUE
data_user = users["password"] != password?
TRUE
raise ValueError "password salah" (eror manual)
FALSE
FALSE
KeyError (username tidak ada)
return(username, data_user["level"]"
End (login berhasil)
except (KeyError, ValueError) Output "Username atau Password salah!"
Input "Tekan Enter untuk lanjut"
End (login berhasil)


Start
hitung_total_rekursif(daftar_kode, indeks=0)
indeks >= len(daftar_kode)?
TRUE
return 0
FALSE
t = transaksi[daftar_kode[indeks]]
t["jenis"] == "Setor"?
TRUE
t["nominal"]
FALSE
-t["nominal"]
return nilai + hitung_total_rekursif(daftar_kode, indeks + 1)
End

Program ini adalah aplikasi pengelolaan data tabungan berbasis terminal yang dibuat dengan Python. Program menyimpan data dalam variabel global berupa dictionary, yaitu users untuk akun pengguna, transaksi untuk data setoran dan penarikan, serta nomor_terakhir dan MIN_PASSWORD sebagai pendukung. Program berjalan terus dalam perulangan while sampai pengguna memilih keluar.

Alur Logika Program

1. Input. Pengguna memilih menu utama, yaitu Login, Register, atau Keluar. Pada Login, pengguna memasukkan username dan password. Pada Register, pengguna memasukkan username dan password baru.

2. Proses autentikasi. Saat login, program mencocokkan username dan password dengan data pada users. Saat register, program memvalidasi bahwa username dan password tidak kosong, username belum terpakai, dan password minimal 4 karakter. Akun baru otomatis berlevel user.

3. Proses setelah login. Program menyaring data sesuai level akun. Admin dapat melihat dan mengelola semua transaksi, sedangkan user biasa hanya dapat mengelola transaksi miliknya sendiri. Setelah itu, pengguna memilih salah satu menu CRUD:

1. Tambah (Create): pengguna memasukkan keterangan, jenis transaksi (Setor/Tarik), dan nominal. Admin juga memasukkan username pemilik. Program memvalidasi input, termasuk memeriksa kecukupan saldo pada penarikan. Jika valid, fungsi buat_kode_transaksi() membuat kode otomatis (TRX002, TRX003, dst.) dan data disimpan bersama tanggal hari ini.
2. Lihat (Read): program menampilkan daftar transaksi melalui prosedur tampilkan_daftar(), lalu menghitung total saldo dengan fungsi rekursif hitung_total_rekursif().
3. Ubah (Update): pengguna memilih nomor transaksi dan bagian yang diubah (keterangan, jenis, atau nominal). Data lama disalin terlebih dahulu. Jika perubahan membuat saldo pemilik menjadi minus, perubahan dibatalkan dan data lama dikembalikan.
4. Hapus (Delete): pengguna memilih nomor transaksi dan mengonfirmasi (y/n). Fungsi hitung_saldo() memeriksa saldo tanpa transaksi tersebut. Data hanya dihapus jika saldo tidak menjadi minus.

4. Penanganan kesalahan. Setiap input divalidasi menggunakan try-except-else-finally untuk menangani ValueError, KeyError, dan IndexError, sehingga program tidak berhenti ketika terjadi kesalahan input.

5. Output. Program menampilkan menu, daftar transaksi, total saldo, serta pesan keberhasilan atau kegagalan pada setiap proses. Setelah logout, pengguna kembali ke menu utama, dan program berakhir ketika memilih Keluar.

2. Deskripsi Singkat Program

Program ini adalah aplikasi pengelolaan data tabungan berbasis terminal yang dibuat dengan Python, di mana pengguna dapat mendaftar, login, lalu mencatat setoran dan penarikan dana. Data disimpan sementara di memori dalam bentuk dictionary (users dan transaksi), sehingga akan kembali ke data awal setiap program dijalankan ulang. Terdapat dua level akun, yaitu admin yang dapat melihat dan mengelola transaksi semua pengguna, serta user biasa yang hanya dapat mengelola transaksi miliknya sendiri, sedangkan akun hasil Register otomatis berlevel user. Program menyediakan operasi CRUD, yaitu menambah transaksi Setor atau Tarik dengan kode otomatis (TRX002, TRX003, dan seterusnya) dan tanggal hari ini, melihat daftar transaksi beserta total saldo, mengubah keterangan, jenis, atau nominal transaksi, serta menghapus transaksi setelah konfirmasi. Setiap input divalidasi, mulai dari input kosong, nominal yang bukan angka positif, hingga username yang sudah dipakai, dan program mencegah saldo menjadi minus saat menarik, mengubah, maupun menghapus transaksi, dengan kesalahan ditangani menggunakan try-except-else-finally agar program tidak berhenti. Secara struktur, program memakai 2 prosedur (bersihkan_layar dan tampilkan_daftar), 4 fungsi (buat_kode_transaksi, login, hitung_saldo, dan hitung_total_rekursif yang menggunakan rekursi), serta 4 variabel global (users, transaksi, nomor_terakhir, dan MIN_PASSWORD).

3. Source Code

A. FITUR REGISTER

Berfungsi Mendaftarkan akun baru dengan level user.

```
if pilihan == "2":
        bersihkan_layar()
        print("=== REGISTER ===")
        username_baru = input("Username baru: ").strip()
        password_baru = input(f"Password (min. {MIN_PASSWORD} karakter):
").strip()
        try:
            if username_baru == "" or password_baru == "":
                raise ValueError("Username atau password tidak boleh kosong!")
            if username_baru in users:
                raise ValueError("Username sudah dipakai!")
            if len(password_baru) < MIN_PASSWORD:
                raise ValueError(f"Password minimal {MIN_PASSWORD} karakter!")
        except ValueError as e:
            print(f"Register gagal: {e}")
        else:
            users[username_baru] = {
                "password" : password_baru,
```

```
                "level" : "user"
            }
            print("Register berhasil, silahkan login.")
```

B. Fitur Login

Memverifikasi username dan password, lalu mengembalikan (username, level) atau None jika gagal.

```
def login():
    bersihkan_layar()
    print("=== LOGIN ===")
    username = input("Username: ").strip()
    password = input("Password: ").strip()
    try:
        data_user = users[username]                # KeyError jika username tidak
ada
        if data_user["password"] != password:
            raise ValueError("password salah")     # error manual
    except (KeyError, ValueError):
        print("Username atau Password salah!")
        input("Tekan Enter untuk lanjut...")
        return None
    else:
        return (username, data_user["level"])
```

C. Fitur Logout

Keluar dari akun dan kembali ke menu utama

```
 elif menu == "5":
                sedang_login = False
```

D. Fitur Tambah Transaksi (Create)

Menambah transaksi setor atai tarik.

```
if menu == "1":
                bersihkan_layar()
                print("=== TAMBAH TRANSAKSI ===")
                if level == "admin":
                    pemilik = input("Username pemilik: ").strip()
                    try:
                        if users[pemilik]["level"] != "user":
                            raise ValueError("Pemilik harus user biasa!")
                    except KeyError:
                        print("Pemilik tidak ditemukan!")
                        input("Tekan Enter untuk lanjut...")
                        continue
                    except ValueError as e:
                        print(e)
                        input("Tekan Enter untuk lanjut...")
                        continue
                else:
                    pemilik = username

                keterangan = input("Keterangan: ").strip()
                print("Jenis transaksi: 1. Setor 2. Tarik")
                jenis_pilih = input("Pilih jenis: ").strip()
                nominal_teks = input("Nominal (Rp): ").strip()

                try:
                    if keterangan == "":
                        raise ValueError("Keterangan tidak boleh kosong!")
                    if jenis_pilih != "1" and jenis_pilih != "2":
                        raise ValueError("Jenis transaksi tidak valid!")
                    if not nominal_teks.isdigit() or int(nominal_teks) <= 0:
                        raise ValueError("Nominal harus angka lebih dari 0!")
                    nominal = int(nominal_teks)
                    saldo = hitung_saldo(pemilik)
                    if jenis_pilih == "2" and nominal > saldo:
                        raise ValueError(f"Saldo tidak cukup! Saldo saat ini
Rp{saldo:,}")
                except ValueError as e:
```

```
                    print(f"Gagal: {e}")
                else:
                    kode = buat_kode_transaksi()
                    jenis = "Setor" if jenis_pilih == "1" else "Tarik"
                    transaksi.update({
```

E. Fitur Lihat Transaksi (Read)

Menampilkan daftar transaksi beserta total saldo.

```
elif menu == "2":
                bersihkan_layar()
                print("=== DATA TRANSAKSI ===")
                if len(tampil) == 0:
                    print("Belum ada data transaksi.")
                else:
                    tampilkan_daftar(tampil)
                    total = hitung_total_rekursif(tampil)
                    print(f"\nTotal saldo: Rp{total:,}")
```

F. Fitur Ubah Transaksi (Update)

Mengubah keterangan, jenis, atau nominal transaksi.

```
lama = transaksi[kode_ubah].copy()  # salinan data lama
                        try:
                            if sub == "1":
                                baru = input("Keterangan baru: ").strip()
                                if baru == "":
                                    raise ValueError("Keterangan tidak boleh
kosong!")
                                transaksi[kode_ubah].update({"keterangan" : baru})
                            elif sub == "2":
                                print("1. Setor 2. Tarik")
                                j = input("Jenis baru: ").strip()
                                if j != "1" and j != "2":
                                    raise ValueError("Jenis transaksi tidak
valid!")
                                transaksi[kode_ubah].update({"jenis" : "Setor" if
j == "1" else "Tarik"})
                            elif sub == "3":
                                n = input("Nominal baru (Rp): ").strip()
                                if not n.isdigit() or int(n) <= 0:
                                    raise ValueError("Nominal harus angka lebih
dari 0!")
                                transaksi[kode_ubah].update({"nominal" : int(n)})
```

```
if hitung_saldo(lama["pemilik"]) < 0:
                                raise ValueError("Perubahan dibatalkan, saldo akan
menjadi minus!")
                        except ValueError as e:
                            transaksi[kode_ubah] = lama     # kembalikan data lama
```

G. Fitur Hapus Transaksi (Delete)

Menghapus transaksi setelah dikonfirmasi.

```
yakin = input("Yakin hapus? (y/n): ").strip().lower()
                        if yakin == "y":
                            pemilik_hapus = transaksi[kode_hapus]["pemilik"]
                            # cek saldo pemilik jika transaksi ini dihapus
                            if hitung_saldo(pemilik_hapus, kode_hapus) < 0:
                                print("Penghapusan dibatalkan, saldo akan menjadi
minus!")
                            else:
                                cache = transaksi.pop(kode_hapus)
                                print(f"Data {kode_hapus} ({cache['keterangan']})
berhasil dihapus.")
```

H. Fitur Hak Akses Berdasarkan Level (Admin vs User)

Membatasi data yang boleh dilihat.

```
tampil = []
            for kode in transaksi.keys():
                if level == "admin" or transaksi[kode]["pemilik"] == username:
                    tampil.append(kode)
```

I. Fitur Transaksi Otomatis

Membuat kode unik berurutan (TRX002, TRX003, dst).

```
def buat_kode_transaksi():
    global nomor_terakhir
    nomor_terakhir += 1
    kode_baru = "TRX" + str(nomor_terakhir).zfill(3)
    return kode_baru
```

J. Fitur Perhitungan Saldo Otomatis

Menghitung saldo, dengan dua cara.

```
def hitung_saldo(pemilik, kecuali_kode=None):
    saldo = 0
    for kode, t in transaksi.items():
        if kode != kecuali_kode and t["pemilik"] == pemilik:
            if t["jenis"] == "Setor":
                saldo += t["nominal"]
            else:
                saldo -= t["nominal"]
    return saldo
def hitung_total_rekursif(daftar_kode, indeks=0):
    # Base case : semua transaksi sudah dihitung
    if indeks >= len(daftar_kode):
        return 0
    # Recursive case : nilai transaksi ini + total transaksi berikutnya
    t = transaksi[daftar_kode[indeks]]
    nilai = t["nominal"] if t["jenis"] == "Setor" else -t["nominal"]
    return nilai + hitung_total_rekursif(daftar_kode, indeks + 1)
```

K. Fitur Penyimpanan Data Menggunakan Nested Dictionary

Menyimpan data pengguna dan transaksi dalam struktur dictionary bersarang (nested dictionary).

```
users = {
    "admin" : {
        "password" : "admin123",
        "level" : "admin"
    },
```

```
    "sartika" : {
        "password" : "1234",
        "level" : "user"
    }
}
transaksi = {
    "TRX001" : {
        "pemilik" : "sartika",
        "tanggal" : "2026-10-14",
        "keterangan" : "Setoran awal",
        "jenis" : "Setor",
        "nominal" : 1250000
    }
}
```

L. Fitur Membersihkan Terminal

Membersihkan layar terminal agar tampilan menu lebih rapi.

```
import os

os.system("cls" if os.name == "nt" else "clear")
```

M. Fitur Keluar dari Program

Mengakhiri program ketika pengguna memilih menu Keluar.

```
elif pilihan == "3":
        print("Terima kasih, program selesai.")
```

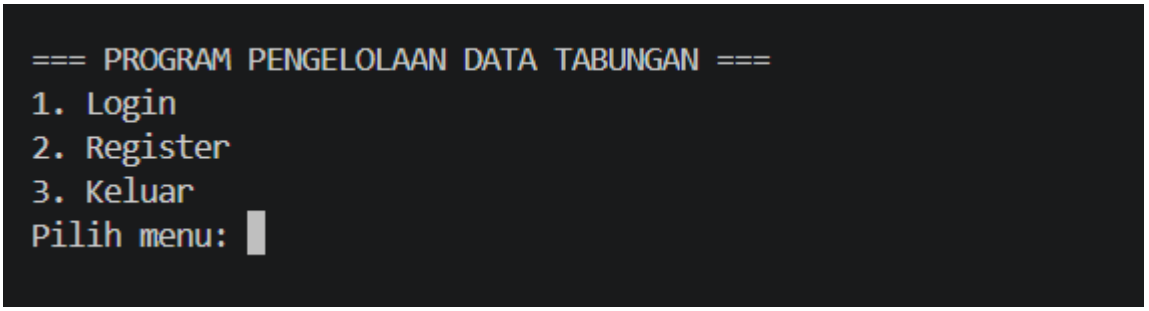
```
        break
```

N. Fitur Tanggal Transaksi Otomatis

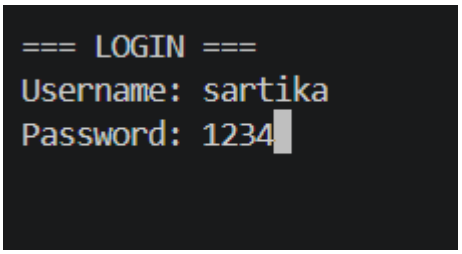
Menyimpan tanggal saat transaksi dibuat secara otomatis menggunakan modul datetime, sehingga pengguna tidak perlu memasukkan tanggal secara manual.

```
from datetime import date
```

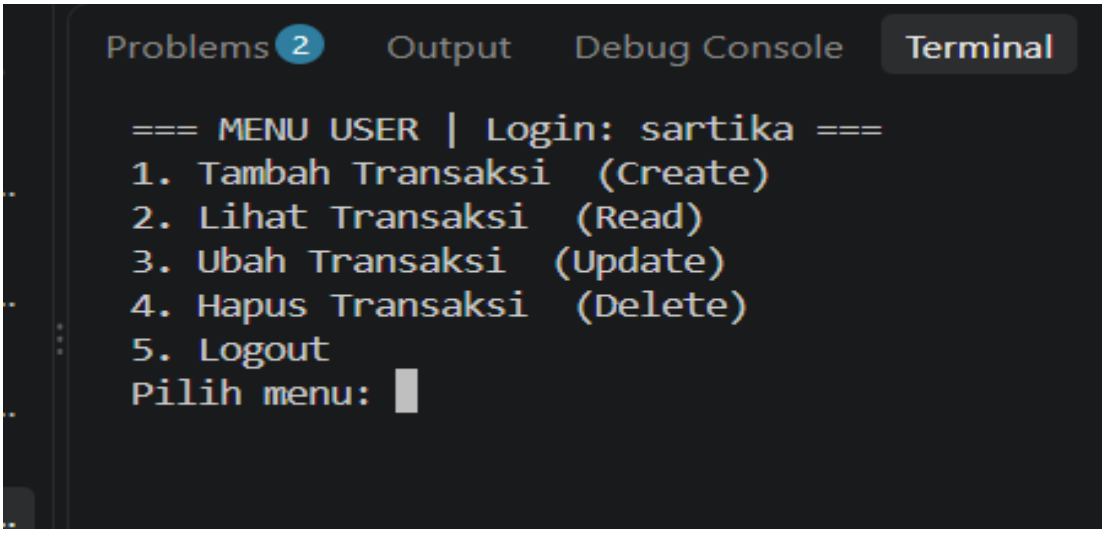
4. Hasil Output



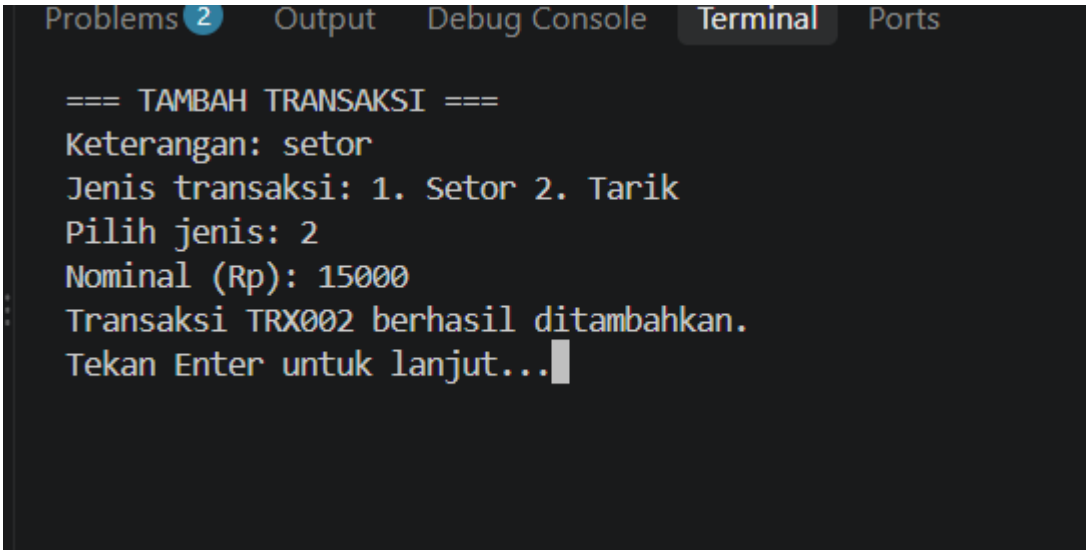
Gambar 4.1



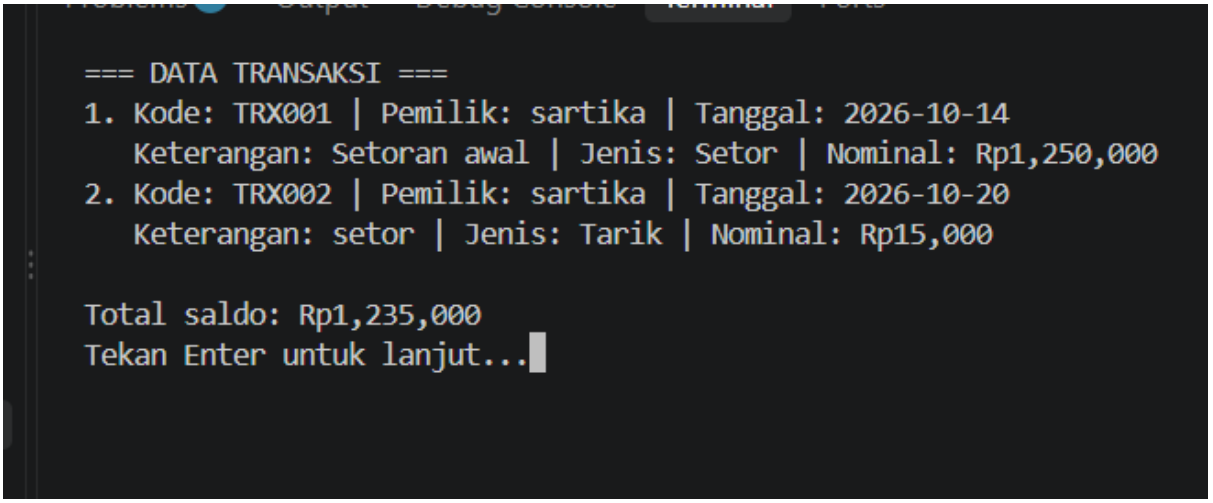
Gambar 4.2



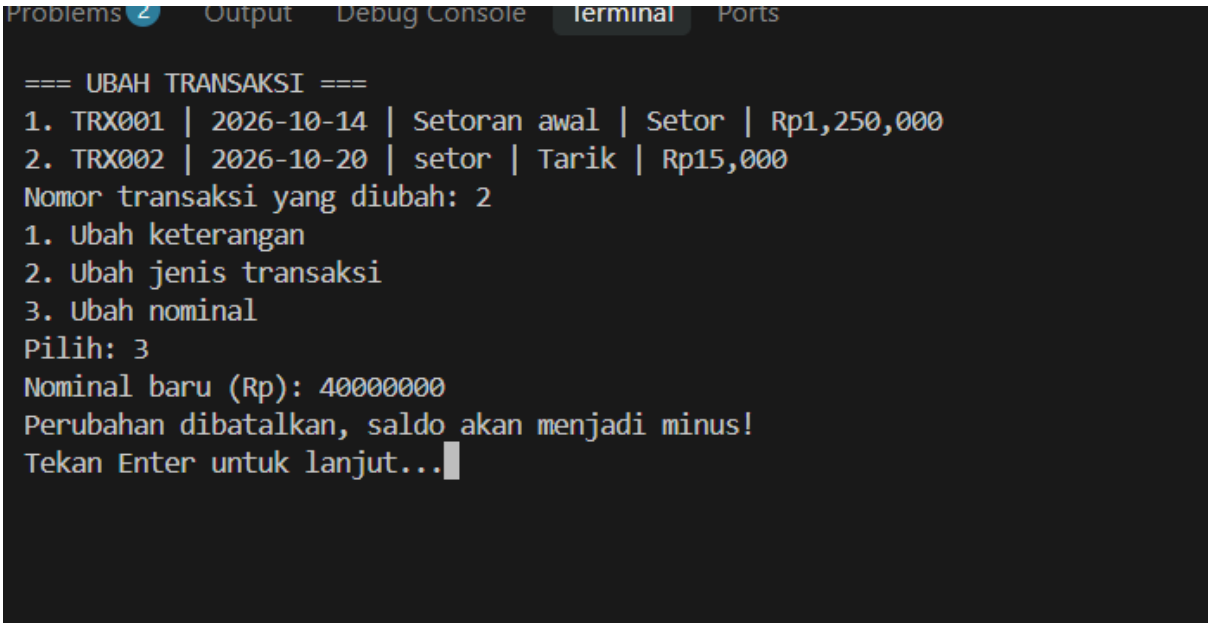
Gambar 4.3



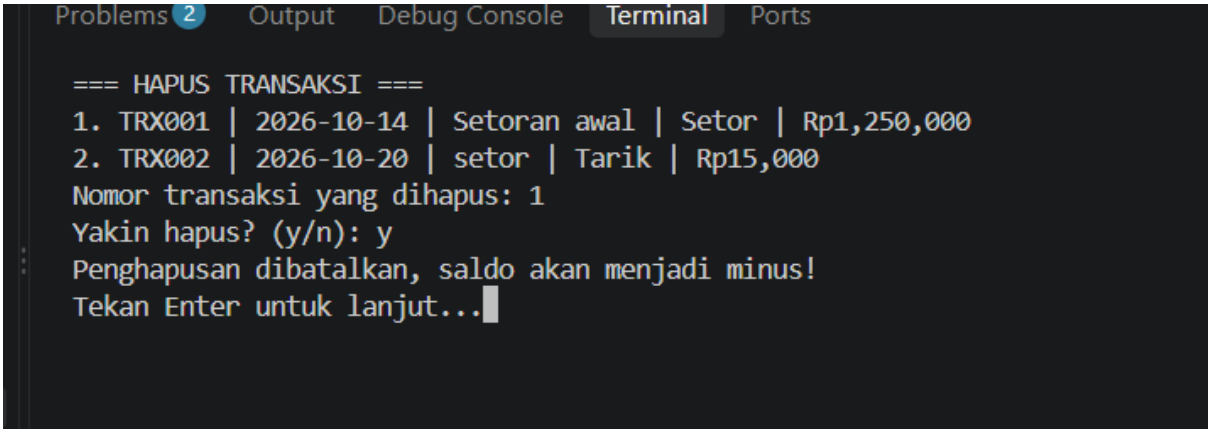
Gambart 4.4



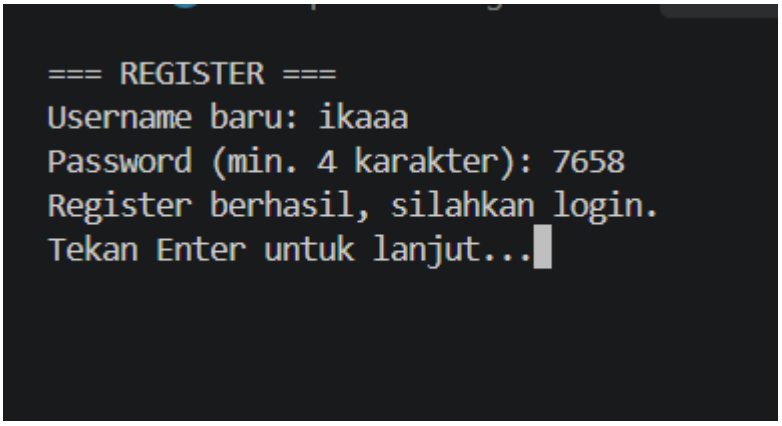
Gambar 4.5



Gambar 4.6



Gambar 4.7



Gambar 4.8

```
=== PROGRAM PENGELOLAAN DATA TABUNGAN ===
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih, program selesai.
PS C:\Users\MSI THIN 15\Downloads\perkuliahan\praktikum_APD_B2'26>
```

Gambar 4.9

5. Langkah-langkah Git

5. 1 Git Add

Berfungsi untuk memasukkan perubahan file yang ingin disimpan ke dalam staging area sebelum di-commit.

```
PS C:\Users\MSI THIN 15\Downloads\perkuliahan\praktikum_APD_B2'26> git add .
```

5.2 Git Commit

Berfungsi untuk menyimpan perubahan yang sudah ada di staging area ke dalam riwayat Git dengan memberikan pesan perubahan.

```
PS C:\Users\MSI THIN 15\Downloads\perkuliahan\praktikum_APD_B2'26> git commit -m "tugas"
[main 71824ca] tugas
 3 files changed, 616 insertions(+)
 create mode 100644 Posttest/posttest-apd-7/2609106058-SARTIKA-WAHIDIYATI-PT-7.py
 create mode 100644 kelas/pertemuan_8/main.py
 create mode 100644 kelas/pertemuan_8/pertemuan8.py
```

5.3 Git Push

Berfungsi untuk mengirim perubahan yang sudah di-commit dari komputer lokal ke repository online, seperti GitHub.

```
PS C:\Users\MSI THIN 15\Downloads\perkuliahan\praktikum_APD_B2'26> git push origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 6.27 KiB | 6.27 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/sartikawahidiyati/Praktikum_APD_B2-26.git
   b3ba428..5d26ea0  main -> main
```